

Clarification: mid vs mid + 1 in binary search for the first B

A question came up about slide 12: when `arr[mid] == 'B'`, shouldn't the recursive call use `mid + 1`?

TL;DR. We were right to use `mid + 1`, but if we do that, we would have to add extra base cases. It is also correct to use `mid` instead of `mid + 1`, **only if** you change the definition to be "return `r` if not found", rather than "return `-1` if not found."

Here is a more in-depth explanation:

What goes wrong with `mid + 1` (end exclusive, `arr[l..r-1]`, return `r` if not found). Try `findFirstB("AB", 0, 2)`, where the answer is 1:

- `mid = 1` and `arr[1] == 'B'`.
- `(l, mid + 1)` gives `(0, 2)`, which is the same call, so it recurses forever.
- The problem is that a range of size 2 has `mid = r - 1`, so `mid + 1 = r` and the range doesn't shrink.

How to make `mid + 1` work. You need base cases for size 2 (and size 1) so the recursion only runs on ranges of size 3 or more. There, `mid + 1 < r`, so the range always shrinks:

```
findFirstB(arr, l, r):
    if l == r: return r
    if r - l == 1: return l if arr[l] == 'B' else r
    if r - l == 2:
        if arr[l] == 'B': return l
        return l + 1 if arr[l+1] == 'B' else r

    mid = (l + r) // 2
    if arr[mid] == 'B':
        return findFirstB(arr, l, mid + 1)    # searches l..mid, guaranteed to find a B
    else:
        return findFirstB(arr, mid + 1, r)
```

Why the slide version is simpler. With `(l, mid)`:

- The range shrinks whenever `r - l >= 1`, so the only base case is `l == r`.
- `(l, mid)` covers only `l..mid-1`, but if no B is found there, it returns `mid`. `arr[mid]` is a B, so that is the right answer.